

HNSW

1.What is HNSW?

HNSW = Hierarchical Navigable Small World

It is a **data structure + algorithm** used for **fast similarity search** (nearest neighbor search) in large datasets.

HNSW helps you find the most similar item very fast in huge data.

2.Where is HNSW Used?

HNSW is used when you have **vectors/embeddings** and you want to find similar ones quickly.

Real-world uses:

AI & Machine Learning

- ChatGPT embeddings search
- Semantic search (Google-like)
- Vector databases (Pinecone, FAISS, Milvus, Weaviate)

Recommendation Systems

- Netflix movies
- Amazon products
- Spotify songs

Image / Face Search

- Find similar images
- Face recognition

3.Why Not Normal Search?

Suppose you have **1 billion vectors**.

To find the closest one:

Brute Force:

Compare with all vectors $\rightarrow O(N)$

Too slow

HNSW:

Search in graph $\rightarrow O(\log N)$ or sub-linear

Very fast

4. Idea Behind HNSW

HNSW is based on **Small World Graph Theory**.

In real life:

- You can reach anyone in the world through a few friends.
- Social networks are “small world”.

HNSW copies this idea.

5. How HNSW Works

HNSW builds **multi-layer graph**:

Layer 3 (very few nodes) $\rightarrow$ fast jump

Layer 2

Layer 1

Layer 0 (all nodes)

Top layer = shortcuts

Bottom layer = detailed search

6. Step-by-Step Working

1. Insert Points into Graph

Each vector = node

Edges connect similar vectors.

2. Multi-Level Hierarchy

When inserting a node:

- Randomly assign a **max layer level**
- Higher level = fewer nodes

Example:

Node A → Layer 3

Node B → Layer 1

Node C → Layer 0

Top layers have **few nodes**, bottom layers have **all nodes**.

7. Search Process (Important)

Step 1: Start from top layer

Pick an entry point (random or last inserted node)

Step 2: Greedy Search

At each layer:

- Move to neighbor that is closest to query
- Stop when no closer neighbor found

Step 3: Go Down One Layer

Use the found node as starting point for next layer.

Step 4: Repeat till Layer 0

Step 5: Final Detailed Search

At layer 0, find exact nearest neighbors.

Imagine searching a house in India:

Country → State → City → Street → House

**Instead of checking every house,
you jump using hierarchy.**

HNSW does same for vectors.

8. Why HNSW is Fast?

1. Graph shortcuts

High-level layers skip large areas.

2. Greedy navigation

Always move closer to target.

3.No full scan

Avoid $O(N)$ search.

Operation	Complexity
Insert	$O(\log N)$
Search	$O(\log N)$ (approx)
Memory	High (graph edges)

Disadvantages

- Uses more memory
- Complex to implement
- Approximate (not always exact nearest)
- But speed is worth it.

Where HNSW is Implemented

Popular Libraries:

- FAISS (Facebook)
- Milvus
- Weaviate
- Pinecone
- nmslib

HNSW vs Other Methods

Method	Speed	Accuracy	Memory
Brute Force	Slow	Exact	Low
KD-Tree	Medium	Medium	Low
LSH	Fast	Low	Medium
HNSW	Very Fast	High	High

9.What is Image Compression?

👉 Image compression = reducing file size by removing redundant or less important data.

There are **two main types**:

1. **Lossless Compression** (no quality loss)
2. **Lossy Compression** (quality loss)

When 10MB Image Becomes 2MB — What Happens Internally?

Step 1: Image is Just Numbers

An image is stored as:

- Pixels
- Each pixel has RGB values
- Example:
 - (255, 0, 0) = Red
 - (0, 255, 0) = Green

A 10MB image = **millions of pixels with precise values**.

10.LOSSY COMPRESSION (Why Quality Reduces)

This is what JPEG, WebP, HEIC mostly use.

1.Color Data Reduction

Human eyes **cannot see tiny color differences**.

So compression:

- Removes very small color variations
- Merges similar colors

Example:

Before:

Pixel1 = (100,101,102)

Pixel2 = (101,100,103)

After compression:

Both become (100,100,100)

File size ↓

Sharpness ↓

2.DCT Transformation

JPEG uses **DCT (Discrete Cosine Transform)**.

What happens:

- Image is converted into frequency components
- Low frequency = important (shapes, colors)
- High frequency = tiny details (texture, noise)

Compression removes **high-frequency data**.

Meaning:

- Skin texture lost
- Hair details blurred
- Edges become soft

3.Quantization

This step **rounds off values**.

Example:

Real value = 23.4567

Stored as = 23

Precision lost

Image becomes blurry

This is why **quality reduces permanently**.

4.Chroma Subsampling

Human eyes see brightness better than color.

So compression:

- Keeps brightness high

- Reduces color resolution

Example:

4:4:4 → 4:2:0

Color bleeding

Edges less sharp

11. LOSSLESS COMPRESSION

Used in PNG, some WebP, ZIP.

Techniques:

1. Run-Length Encoding (RLE)

If many pixels same:

White White White White White

→ White × 5

2. Huffman Coding

Common colors get shorter codes.

3. LZ77 / LZ78 Dictionary

Repeated patterns stored once.

File size reduces but **not as much as lossy**.

Why 10MB → 2MB Reduces Quality?

Because:

1. Pixel information deleted
2. Fine details removed
3. Color precision reduced
4. Math rounding error Deleted data can NEVER be recovered.

12. Why Companies Compress Images?

Faster Loading

2MB loads 5× faster than 10MB.

Less Storage Cost

CDN storage is expensive.

Bandwidth Saving

Mobile users save data.

“Lossy compression reduces file size by discarding high-frequency and perceptually insignificant information using transforms like DCT and quantization, which leads to irreversible quality loss.”

Format	Compression	Quality
JPEG	Lossy	Medium
PNG	Lossless	High
WebP	Both	Better than JPEG
AVIF	Advanced	Best compression

Real Engineering Example

Google compresses images because:

1MB saved per user × 1B users = 1 PB bandwidth saved

= Millions of dollars saved



13.What Is This Image About?

This is **Approach 4: Compression Method – Product Quantization (PQ)**

PQ is used to **compress big vectors (embeddings)** so they use very less RAM.

Used in:

- FAISS
- Vector Databases
- Search engines
- AI systems (ChatGPT embeddings, Google search)

Problem: Why We Need PQ

Example in Image:

Vector size = 1536 dimensions

Each value = 32-bit float = 4 bytes

Memory for 1 vector:

$1536 \times 4 \text{ bytes} = 6144 \text{ bytes} \approx 6 \text{ KB}$

If you have 1 BILLION vectors:

$6 \text{ KB} \times 1,000,000,000 = 6 \text{ TB RAM}$

Impossible to store in RAM.

Solution: Product Quantization (PQ)

Idea:

Instead of storing real float numbers, store **small integer codes**.

Step 1: Split Big Vector into Chunks

Image says:

It chops the big vector into smaller segments

Example vector:

$V = [1.1, 2.3, 0.9, 3.4, 8.8, 7.6, 9.1, \dots]$

Split into chunks:

$\text{chunk1} = [1.1, 2.3, 0.9, 3.4]$

$\text{chunk2} = [8.8, 7.6, 9.1, 6.5]$

$\text{chunk3} = [\dots]$

$\text{chunk4} = [\dots]$

If vector has 1536 dims

You make 128 chunks of 12 dims each (example).

Step 2: Create Codebooks (Clusters)

Image says:

Create Codebook (cluster / centroid)

For each chunk:

Run **K-Means clustering**.

Example:

For chunk1, create 256 clusters:

0 → centroid [1.2, 3.6, 4.2, 1.1]

1 → centroid [2.2, 2.3, 5.7, 1.1]

...

255 → centroid [...]

These are **representative vectors**.

Step 3: Replace Real Values with Small Codes

Instead of storing:

[1.1, 2.3, 0.9, 3.4]

Store only:

code = 42

Meaning:

chunk1 ≈ centroid #42

Memory Reduction MAGIC

Before:

Each chunk = 4 floats = 16 bytes

After PQ:

Only store **1 byte code (0–255)**

Memory Example from Image

Original:

1536 dims × 4 bytes = 6144 bytes

After PQ:

1536 dims → 128 chunks → 128 bytes total

6144 bytes → 128 bytes (50x compression)

Step 4: Searching with PQ

When query comes:

1. Split query vector into chunks
2. Compare with centroids
3. Use precomputed distance tables
4. Sum distances

No need to decode full vectors.

SUPER FAST.

Why Quality Reduces?

Because:

- We replace real floats with nearest centroid
- It is approximation

Example:

Real value = [1.1, 2.3, 0.9, 3.4]

Stored centroid = [1.2, 2.1, 1.0, 3.3]

Slight error introduced.

Simple Analogy

Original:

Exact sentence:

“I am going to school today at 9:15 AM”

PQ:

Store short code:

Sentence type #42

Meaning similar but not exact.

Why PQ Is Used in Big Tech

Because:

Method	Memory	Speed
Raw vectors	Huge	Slow
HNSW only	Huge	Fast
HNSW + PQ	Low	Fast

This combo is used in FAISS.

“Product Quantization compresses high-dimensional vectors by splitting them into sub-vectors and replacing each sub-vector with the index of its nearest centroid, reducing memory and enabling fast approximate distance computation.”

Why Called PRODUCT Quantization?

Because:

Total compression = Quantization of subspaces × all chunks

Product of quantizers.

Drawbacks

- Approximate results
- Training clusters expensive
- Accuracy slightly lower than raw vectors



14.Which Method is BEST?

The top part compares 3 vector indexing methods:

Methods Compared

- 1. HNSW
- 2. IVF (Standalone)
- 3. Trees (KD-Tree, etc.)

SPEED (Query Latency)

Method	Speed
HNSW	Highest (fastest)
IVF	Medium
Trees	Low (bad in high dimensions)

Why?

- HNSW uses graph shortcuts

- IVF searches clusters
- Trees break in high-dimensional data (curse of dimensionality)

Accuracy (Recall)

Method	Accuracy
HNSW	Highest
IVF	Medium
Trees	Low

HNSW gives best nearest neighbors.

Memory Usage

Method	Memory
HNSW	Highest (graph edges)
IVF	Low
Trees	Medium

HNSW fast but RAM heavy

IVF slower but RAM friendly

Golden System Design Insight

HNSW = Speed + Accuracy

IVF = Memory efficient

Trees = Old method, not good for AI embeddings

15.Vector Databases / Libraries Comparison

This section lists **real vector DBs and what indexing they use internally.**

Milvus

- Category: Native vector DB
- Feature: Open-source, scalable, decoupled storage + compute
- Indexing:
 - HNSW
 - IVF-PQ
 - IVF-Flat

Used in enterprises.

Pinecone

- Managed SaaS vector DB
- You don't manage infra
- Indexing: **HNSW**

Used in startups & AI apps.

Weaviate

- Open-source vector DB
- GraphQL API
- Knowledge graph support
- Indexing: **HNSW**

Qdrant

- Written in Rust
- High performance
- Advanced filtering
- Indexing: **HNSW + custom quantization**

PostgreSQL (pgvector)

- Integrated with SQL
- Store vectors + relational data
- Indexing: IVF-Flat, HNSW

Good for traditional DB users.

Elasticsearch

- Keyword + vector search
- Indexing: **HNSW**

Redis

- In-memory DB
- Ultra fast
- Vector search built-in
- Indexing: **HNSW**

FAISS (Meta)

- Not a DB, a library
- Base engine behind many DBs
- Indexing: HNSW, IVF, PQ, etc.

FAISS is like Linux kernel of vector search.

16.What Data is Actually Stored in Vector DB?

This is MOST IMPORTANT.

Example Stored Record

```
my_id = "product_456"
```

```
my_vector = [0.98, 0.23, -0.11, ...]
```

```
my_metadata = {
```

```
  "product_name": "Red Running Shoes",
```

```
  "price": 89.99
```

```
}
```

So Vector DB Stores 3 Things

1.ID

Unique key (like primary key)

2. Vector (Embedding)

Huge float array:

```
[0.98, 0.23, -0.11, ...]
```

This is AI meaning representation.

3.Metadata

Extra data:

- name
- price
- tags
- category
- user info

Used for filtering.

17. Word/Sentence → Vector Diagram

Bottom diagram:

Word / Sentence → Vector

Meaning:

Text:

"Red running shoes"

AI model converts to vector:

[0.98, 0.23, -0.11, ...]

This vector stored in DB.

How Search Works

User query:

"best shoes for running"

Steps:

1. Convert query → vector
2. Search nearest vectors using HNSW/IVF
3. Return metadata (product info)

Big Picture System Flow

Text/Image → Embedding Model → Vector

Vector → Vector Database

Query → Vector → ANN Search → Results

1.HNSW

Graph-based ANN index with high recall and low latency.

2.IVF

Cluster-based ANN index optimized for memory efficiency.

3.Vector DB stores

ID + embedding vector + metadata.